Kalman Filter Theory and Applications
Heuristic Overview

Michael L. Carroll

July 11, 2020

©2020 by Michael L. Carroll

Part I
The Basic Kalman Equations
Topics

- Understanding the Equations: Heuristic Introduction

- Understanding the Equations: Heuristic Introduction
- Equation Drilldown: Taking the Equations Apart

- Understanding the Equations: Heuristic Introduction
- Equation Drilldown: Taking the Equations Apart
- State Space Concepts

- Understanding the Equations: Heuristic Introduction
- Equation Drilldown: Taking the Equations Apart
- State Space Concepts

- **Recursive Predictor-Corrector Algorithms**

- **Recursive Predictor-Corrector Algorithms**
- Running Averages

- **Recursive Predictor-Corrector Algorithms**
- Running Averages
- Prediction (Extrapolation)

- **Recursive Predictor-Corrector Algorithms**
- Running Averages
- Prediction (Extrapolation)
- Correction (Measurement Update)

- **Recursive Predictor-Corrector Algorithms**
- Running Averages
- Prediction (Extrapolation)
- Correction (Measurement Update)
- Gain Computation

- **Recursive Predictor-Corrector Algorithms**
- Running Averages
- Prediction (Extrapolation)
- Correction (Measurement Update)
- Gain Computation

Recursive Predictor-Corrector Algorithm

What kinds of problems are we trying to solve?

Recursive Predictor-Corrector Algorithm

What kinds of problems are we trying to solve?

- Problem: To estimate quantities associated with a noisy dynamic process using imperfect, noisy measurements

Recursive Predictor-Corrector Algorithm

What kinds of problems are we trying to solve?

- Problem: To estimate quantities associated with a noisy dynamic process using imperfect, noisy measurements
  - Convenient to bundle these quantities into a single vector: the **state vector** of the system or process

Recursive Predictor-Corrector Algorithm

What kinds of problems are we trying to solve?

- Problem: To estimate quantities associated with a noisy dynamic process using imperfect, noisy measurements
  - Convenient to bundle these quantities into a single vector: the **state vector** of the system or process
  - State vector changes with time; models are not perfect and have noisy inputs (forcing functions)

Recursive Predictor-Corrector Algorithm

What kinds of problems are we trying to solve?

- Problem: To estimate quantities associated with a noisy dynamic process using imperfect, noisy measurements
  - Convenient to bundle these quantities into a single vector: the **state vector** of the system or process
  - State vector changes with time; models are not perfect and have noisy inputs (forcing functions)
  - Can never really know the true state at any given time – that's why we need an estimator!

Recursive Predictor-Corrector Algorithm

What kinds of problems are we trying to solve?

- Problem: To estimate quantities associated with a noisy dynamic process using imperfect, noisy measurements
  - Convenient to bundle these quantities into a single vector: the **state vector** of the system or process
  - State vector changes with time; models are not perfect and have noisy inputs (forcing functions)
  - Can never really know the true state at any given time – that's why we need an estimator!
- Measurements have their own challenges

Recursive Predictor-Corrector Algorithm

What kinds of problems are we trying to solve?

- Problem: To estimate quantities associated with a noisy dynamic process using imperfect, noisy measurements
  - Convenient to bundle these quantities into a single vector: the **state vector** of the system or process
  - State vector changes with time; models are not perfect and have noisy inputs (forcing functions)
  - Can never really know the true state at any given time – that's why we need an estimator!
- Measurements have their own challenges
  - Arrive sequentially, possibly event-driven

Recursive Predictor-Corrector Algorithm

What kinds of problems are we trying to solve?

- Problem: To estimate quantities associated with a noisy dynamic process using imperfect, noisy measurements
  - Convenient to bundle these quantities into a single vector: the **state vector** of the system or process
  - State vector changes with time; models are not perfect and have noisy inputs (forcing functions)
  - Can never really know the true state at any given time – that's why we need an estimator!
- Measurements have their own challenges
  - Arrive sequentially, possibly event-driven
  - Out of synch with each other and with the estimation period

Recursive Predictor-Corrector Algorithm

What kinds of problems are we trying to solve?

- Problem: To estimate quantities associated with a noisy dynamic process using imperfect, noisy measurements
  - Convenient to bundle these quantities into a single vector: the **state vector** of the system or process
  - State vector changes with time; models are not perfect and have noisy inputs (forcing functions)
  - Can never really know the true state at any given time – that's why we need an estimator!
- Measurements have their own challenges
  - Arrive sequentially, possibly event-driven
  - Out of synch with each other and with the estimation period
  - Generated by multiple sensors with differing accuracies and noise characteristics

Recursive Predictor-Corrector Algorithm

What kinds of problems are we trying to solve?

- Problem: To estimate quantities associated with a noisy dynamic process using imperfect, noisy measurements
  - Convenient to bundle these quantities into a single vector: the **state vector** of the system or process
  - State vector changes with time; models are not perfect and have noisy inputs (forcing functions)
  - Can never really know the true state at any given time – that's why we need an estimator!
- Measurements have their own challenges
  - Arrive sequentially, possibly event-driven
  - Out of synch with each other and with the estimation period
  - Generated by multiple sensors with differing accuracies and noise characteristics
  - Don't necessarily measure the states directly (need a measurement model to relate measurements to the states)

Recursive Predictor-Corrector Algorithm

What kinds of problems are we trying to solve?

- Problem: To estimate quantities associated with a noisy dynamic process using imperfect, noisy measurements
  - Convenient to bundle these quantities into a single vector: the **state vector** of the system or process
  - State vector changes with time; models are not perfect and have noisy inputs (forcing functions)
  - Can never really know the true state at any given time – that's why we need an estimator!
- Measurements have their own challenges
  - Arrive sequentially, possibly event-driven
  - Out of synch with each other and with the estimation period
  - Generated by multiple sensors with differing accuracies and noise characteristics
  - Don't necessarily measure the states directly (need a measurement model to relate measurements to the states)

Recursive Predictor-Corrector Algorithm

Recursive Predictor-Corrector Algorithm

- Goal: Devise a method to combine all the measurement information and produce the best estimate of the state of system

Recursive Predictor-Corrector Algorithm

- Goal: Devise a method to combine all the measurement information and produce the best estimate of the state of system
  - Generate estimates in real-time, if possible, and avoid growing memory consumption

Recursive Predictor-Corrector Algorithm

- Goal: Devise a method to combine all the measurement information and produce the best estimate of the state of system
  - Generate estimates in real-time, if possible, and avoid growing memory consumption
- Seek an algorithm to:

Recursive Predictor-Corrector Algorithm

- Goal: Devise a method to combine all the measurement information and produce the best estimate of the state of system
  - Generate estimates in real-time, if possible, and avoid growing memory consumption
- Seek an algorithm to:
  - Predict how the system is changing over time between measurements

Recursive Predictor-Corrector Algorithm

- Goal: Devise a method to combine all the measurement information and produce the best estimate of the state of system
  - Generate estimates in real-time, if possible, and avoid growing memory consumption
- Seek an algorithm to:
  - Predict how the system is changing over time between measurements
  - Correct our predicted state estimates with new measurement

Recursive Predictor-Corrector Algorithm

- Goal: Devise a method to combine all the measurement information and produce the best estimate of the state of system
  - Generate estimates in real-time, if possible, and avoid growing memory consumption
- Seek an algorithm to:
  - Predict how the system is changing over time between measurements
  - Correct our predicted state estimates with new measurement
  - Avoid growing memory requirements
- Bottom Line: We seek a **recursive** predictor-corrector algorithm

Recursive Predictor-Corrector Algorithm

- Goal: Devise a method to combine all the measurement information and produce the best estimate of the state of system
  - Generate estimates in real-time, if possible, and avoid growing memory consumption
- Seek an algorithm to:
  - Predict how the system is changing over time between measurements
  - Correct our predicted state estimates with new measurement
  - Avoid growing memory requirements
- Bottom Line: We seek a **recursive** predictor-corrector algorithm

---

Recursive Predictor-Corrector Algorithm

Recursive Predictor-Corrector Algorithm

- Prediction:

Recursive Predictor-Corrector Algorithm

- Prediction:
  - **Extrapolate** solution from previous iteration to present time

Recursive Predictor-Corrector Algorithm

- Prediction:
  - **Extrapolate** solution from previous iteration to present time (This is how we take system dynamics into account)

Recursive Predictor-Corrector Algorithm

- Prediction:
  - **Extrapolate** solution from previous iteration to present time (This is how we take system dynamics into account)
- Correction:

Recursive Predictor-Corrector Algorithm

- Prediction:
  - **Extrapolate** solution from previous iteration to present time (This is how we take system dynamics into account)
- Correction:
  - Use measurement information to **update** or **correct** the prediction

Recursive Predictor-Corrector Algorithm

- Prediction:
  - **Extrapolate** solution from previous iteration to present time (This is how we take system dynamics into account)
- Correction:
  - Use measurement information to **update** or **correct** the prediction
- And the cycle recursively begins anew!

Recursive Predictor-Corrector Algorithm

- Prediction:
  - **Extrapolate** solution from previous iteration to present time (This is how we take system dynamics into account)
- Correction:
  - Use measurement information to **update** or **correct** the prediction
- And the cycle recursively begins anew!

- Time-dependent scalar or vector quantities: $x(k)$ (discrete), $x(t)$ (continuous)

- Time-dependent scalar or vector quantities: $x(k)$ (discrete), $x(t)$ (continuous)
- Corresponding vector components: $x_i(k)$ or $x_i(t)$

- Time-dependent scalar or vector quantities:
  $x(k)$ (discrete), $x(t)$ (continuous)
- Corresponding vector components:
  $x_i(k)$ or $x_i(t)$
- Time-dependent matrix quantities (capital letters):
  $P(k)$ (discrete), $P(t)$ (continuous)

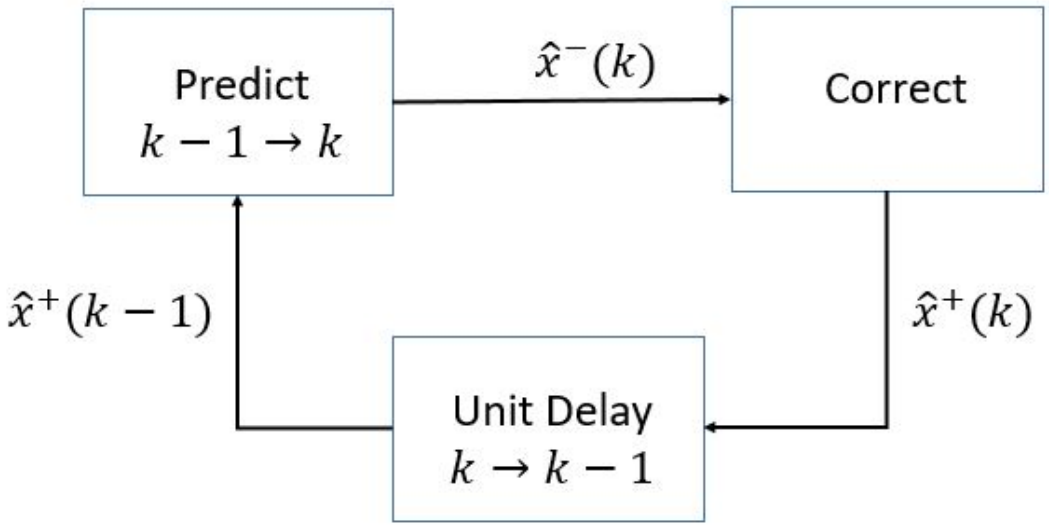
- Time-dependent scalar or vector quantities:
  $x(k)$ (discrete), $x(t)$ (continuous)
- Corresponding vector components:
  $x_i(k)$ or $x_i(t)$
- Time-dependent matrix quantities (capital letters):
  $P(k)$ (discrete), $P(t)$ (continuous)
- Corresponding matrix components (elements):
  $p_{ij}(k)$ or $p_{ij}(t)$

- Time-dependent scalar or vector quantities:
  $x(k)$ (discrete), $x(t)$ (continuous)
- Corresponding vector components:
  $x_i(k)$ or $x_i(t)$
- Time-dependent matrix quantities (capital letters):
  $P(k)$ (discrete), $P(t)$ (continuous)
- Corresponding matrix components (elements):
  $p_{ij}(k)$ or $p_{ij}(t)$
- Use hats for estimated quantities and special superscripts to indicate pre- and post-update values:
  $\hat{x}^+(k)$ and $P^-(k)$

- Time-dependent scalar or vector quantities:
  $x(k)$ (discrete), $x(t)$ (continuous)
- Corresponding vector components:
  $x_i(k)$ or $x_i(t)$
- Time-dependent matrix quantities (capital letters):
  $P(k)$ (discrete), $P(t)$ (continuous)
- Corresponding matrix components (elements):
  $p_{ij}(k)$ or $p_{ij}(t)$
- Use hats for estimated quantities and special superscripts to indicate pre- and post-update values:
  $\hat{x}^+(k)$ and $P^-(k)$
- Use tildes for error quantities, i.e., the difference between the estimate and the truth:
  $\tilde{x}(k) := \hat{x}(k) - x(k)$

- Time-dependent scalar or vector quantities:
  $x(k)$ (discrete), $x(t)$ (continuous)
- Corresponding vector components:
  $x_i(k)$ or $x_i(t)$
- Time-dependent matrix quantities (capital letters):
  $P(k)$ (discrete), $P(t)$ (continuous)
- Corresponding matrix components (elements):
  $p_{ij}(k)$ or $p_{ij}(t)$
- Use hats for estimated quantities and special superscripts to indicate pre- and post-update values:
  $\hat{x}^+(k)$ and $P^-(k)$
- Use tildes for error quantities, i.e., the difference between the estimate and the truth:
  $\tilde{x}(k) := \hat{x}(k) - x(k)$

Recursive Predictor-Corrector Algorithm (Discrete System)



Recursive Predictor-Corrector Algorithm

Recursive Predictor-Corrector Algorithm

- Algorithm

Recursive Predictor-Corrector Algorithm

- Algorithm
  - Computational procedure

Recursive Predictor-Corrector Algorithm

- Algorithm
  - Computational procedure
  - Given initial values of some quantities, compute new quantities (using system dynamics and measurement info)

Recursive Predictor-Corrector Algorithm

- Algorithm
  - Computational procedure
  - Given initial values of some quantities, compute new quantities (using system dynamics and measurement info)
- Recursive

Recursive Predictor-Corrector Algorithm

- Algorithm
  - Computational procedure
  - Given initial values of some quantities, compute new quantities (using system dynamics and measurement info)
- Recursive
  - New quantities are same type as the previous

Recursive Predictor-Corrector Algorithm

- Algorithm
  - Computational procedure
  - Given initial values of some quantities, compute new quantities (using system dynamics and measurement info)
- Recursive
  - New quantities are same type as the previous
  - New quantities become initial values for next cycle

Recursive Predictor-Corrector Algorithm

- Algorithm
  - Computational procedure
  - Given initial values of some quantities, compute new quantities (using system dynamics and measurement info)
- Recursive
  - New quantities are same type as the previous
  - New quantities become initial values for next cycle

- Where are we?

- Where are we?
  - Introduced concept of a Predictor-Corrector algorithm

- Where are we?
  - Introduced concept of a Predictor-Corrector algorithm
  - Discussed need for recursive implementation

- Where are we?
  - Introduced concept of a Predictor-Corrector algorithm
  - Discussed need for recursive implementation
- What's next?

- Where are we?
  - Introduced concept of a Predictor-Corrector algorithm
  - Discussed need for recursive implementation
- What's next?
  - In the next video we will look at a simple example

- Where are we?
  - Introduced concept of a Predictor-Corrector algorithm
  - Discussed need for recursive implementation
- What's next?
  - In the next video we will look at a simple example
  - *Recursive Running Average* and related algorithms

- Where are we?
  - Introduced concept of a Predictor-Corrector algorithm
  - Discussed need for recursive implementation
- What's next?
  - In the next video we will look at a simple example
  - *Recursive Running Average* and related algorithms
